Math for Data Science - Lab 10

Course: Math for Data Science

Matrix Algebra and Calculus: An Application to Linear Regression

Today we will practice matrix algebra and calculus with an application that you will get to know very well: **linear regression**. For now we will put aside many (very important) issues around causal and statistical inference, and just think of regression as a predictive or descriptive exercise: what is the best linear model that we can fit to our data?

Exercises 1 and 2 are the core of the lab; Exercises 3 and 4 are optional.

Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Using R instead of Python (optional)

The lab is written in Python, but you can write your answers in R inside this same notebook. This needs R 4.5 or newer, and Jupyter started with rpy2 added: `uv run --with rpy2 jupyter lab`. The `README.md` of the course folder has the details and what to do when it does not work.

If you want to use R instead of Python, uncomment and run the following cell:

```
# %load_ext rpy2.ipython
```

Then, in any subsequent code cell, add `%%R` as the first line to run that cell as R:

```
%%R
x <- rbinom(1000, size = 10, prob = 0.3)
hist(x)
```

Keep running the given Python cells (imports, data loading) as they are. The hints name Python functions, such as `rng.binomial`; you will need to find the R equivalent (`rbinom`) yourself.

Exercise 1: The Least-Squares Estimator

To make things concrete, let's use a real dataset of median home values in Boston. Load `boston_housing.csv` and take a look at the data. What are the included variables? What is the unit of analysis?

A note on this dataset. The Boston housing data (Harrison & Rubinfeld, 1978) is a classic teaching dataset, but the full version includes a variable defined in terms of the racial composition of each town - which is why `scikit-learn` removed it from its distribution. We use only the columns this lab needs, and that variable is not among them.

```
boston = pd.read_csv("boston_housing.csv")
boston.head()
```

Now let's write down our linear model. We are interested in modeling the median home price in a Census tract (`medv`) as a function of the following variables:

- `crim`: per capita crime rate in the town
- `chas`: Charles River dummy variable (1 if tract is on the water, 0 if not)
- `nox`: nitric oxides concentration (parts per 10 million)
- `dis`: weighted distances to five Boston employment centres
- `ptratio`: pupil-teacher ratio by town
- `rad`: index of accessibility to radial highways

We require that our model be **linear in the parameters**, which means that for any tract i we can express the *response* y_i (the median home value) as a *linear combination* of the variables plus some error term:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + ... + \beta_p x_{ip} + \varepsilon_i \qquad (1)$$

Our goal is to find the "best" values of $\beta_0, \beta_1, ..., \beta_p$, where "best" has a specific meaning: the ones that **minimize the sum of squared errors** over the dataset.

Why minimize this quantity and not something else? The short answer is that it has some nice statistical properties. But there are many other *loss functions* you might minimize, which you will meet later in this class and in Machine Learning.

$$\min_{\beta_0,...,\beta_p} \sum_{i=1}^{n} \varepsilon_i^2 = \min_{\beta_0,...,\beta_p} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_{i1} - \beta_2 x_{i2} - ... - \beta_p x_{ip})^2 \qquad (2)$$

Before we go on, let's set this up as an optimization problem in the way we are familiar with: as a system of equations to solve.

$$
\begin{aligned}
\frac{\partial}{\partial \beta_0} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_{i1} - ... - \beta_p x_{ip})^2 = 0 \\
\frac{\partial}{\partial \beta_1} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_{i1} - ... - \beta_p x_{ip})^2 = 0 \\
\frac{\partial}{\partial \beta_2} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_{i1} - ... - \beta_p x_{ip})^2 = 0 \\
\vdots \\
\frac{\partial}{\partial \beta_p} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_{i1} - ... - \beta_p x_{ip})^2 = 0
\end{aligned}
$$

But we won't go this route today. Instead we'll solve the same problem using linear algebra and calculus. Once you know the basic rules, the matrix route is the more efficient one.

Let's write what is represented by Equation (2) above in matrix form:

$$
\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & x_{12} & ... & x_{1p} \\ 1 & x_{21} & x_{22} & ... & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & ... & x_{np} \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix} \qquad (3)
$$

Let's collapse this into

$$
\mathbf{Y} = \mathbf{X}\beta + \varepsilon \qquad (4)
$$

where

- $\mathbf{Y}$ $(n \times 1)$ is called the **response vector**
- $\mathbf{X}$ $(n \times p + 1)$ is called the **design matrix**
- β $(p + 1 \times 1)$ is our vector of **coefficients**
- ε $(n \times 1)$ is our vector of **error terms**

Now let's write the sum of squared error minimization problem in matrix form:

$$
\sum_{i=1}^{n} \varepsilon_i^2 = \begin{bmatrix} \varepsilon_1 & \varepsilon_2 & ... & \varepsilon_n \end{bmatrix} \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix} = \varepsilon^\top \varepsilon \qquad (5)
$$

And now we can do some matrix manipulation:

$$
\begin{aligned}
\varepsilon^\top \varepsilon &= (\mathbf{Y} - \mathbf{X}\beta)^\top (\mathbf{Y} - \mathbf{X}\beta) \\
&= \mathbf{Y}^\top \mathbf{Y} - \mathbf{Y}^\top \mathbf{X}\beta - \beta^\top \mathbf{X}^\top \mathbf{Y} + \beta^\top \mathbf{X}^\top \mathbf{X}\beta \qquad (6)
\end{aligned}
$$

We'll still take the first derivative and set it equal to 0 to solve for our error-minimizing β's, but we'll do it with the whole vector of β's in one shot:

$$\frac{\partial}{\partial \beta}(\mathbf{Y}^\top \mathbf{Y} - \mathbf{Y}^\top \mathbf{X}\beta - \beta^\top \mathbf{X}^\top \mathbf{Y} + \beta^\top \mathbf{X}^\top \mathbf{X}\beta) \qquad (7)$$

1. Use the Matrix Cookbook (Section 2.4, Derivatives of Matrices, Vectors and Scalar Forms) to show that Equation (7) reduces to

$$-2\mathbf{X}^\top \mathbf{Y} + 2\mathbf{X}^\top \mathbf{X}\beta \qquad (8)$$

here your solution

2. Set Equation (8) equal to 0 and solve for $\hat{\beta}$. Show that

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{Y} \qquad (9)$$

here your solution

Exercise 2: Fitting the Boston Housing Data

1. Using the Boston housing data, create your design matrix and response vector.

> **Hint:** your design matrix needs to be a NumPy array, not a data frame. Don't forget the column of 1s for the intercept.

```
# here your code
```

2. Manually compute the $\hat{\beta}$ vector that you derived in Equation (9).

> **Hint:** `np.linalg.inv` computes a matrix inverse, and `@` is matrix multiplication.

```
# here your code
```

3. Check your answer using NumPy's built-in least-squares solver, `np.linalg.lstsq`.

```
# here your code
```

4. Using Equation (4) above, compute the predicted values $\hat{\mathbf{Y}} = \mathbf{X}\hat{\beta}$. Compare them to the predictions from the `np.linalg.lstsq` coefficients.

```
# here your code
```

5. Plot the predicted values against the observed values. What does a perfect model look like on this plot? What do you notice about this one?

```
# here your code
```

here your solution

Exercise 3: Putting the Predictors on the Same Footing

[optional]

Look back at the coefficients from Exercise 2, question 2. Each predictor is measured in its own unit: `crim` is a rate per capita, `nox` is in parts per 10 million, `chas` is a 0/1 dummy, `ptratio` counts pupils per teacher. A coefficient is "the change in `medv` per one *unit* of that predictor", so the coefficients are not comparable with one another. The `nox` coefficient is large partly because `nox` itself only varies over a narrow range - one unit of `nox` is an enormous change.

It is common practice to bring all predictors onto the same footing by **standardizing** them: subtract each column's mean and divide by its standard deviation,

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}, \qquad s_j = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)^2}$$

Every standardized column has mean 0 and standard deviation 1, and its coefficient reads as "the change in `medv` per one *standard deviation* of that predictor". Write the model on the standardized predictors as

$$y_i = \gamma_0 + \gamma_1 z_{i1} + \ldots + \gamma_p z_{ip} + \varepsilon_i$$

using γ to keep its coefficients apart from the β of the original model.

1. *[optional]* Standardize the six predictors, build a new design matrix from them (again with a column of 1s for the intercept), and compute $\hat{\gamma}$ with Equation (9).

> **Hint:** NumPy's `std` divides by n by default; pass `ddof=1` to divide by $n-1$. Either convention works, as long as you convert back in question 3 with the same s_j you standardized with: dividing by n only shrinks every standardized coefficient by the same factor $\sqrt{(n-1)/n}$.

```
# here your code
```

2. *[optional]* Compare $\hat{\gamma}$ with your $\hat{\beta}$ from Exercise 2, question 2. Rank the predictors by the size of their coefficient (ignoring the sign) in each model: does the ranking change? The intercept $\hat{\gamma}_0$ should look familiar - what is it, and why?

here your solution

3. *[optional]* Standardizing changes the units of the coefficients, not the fitted model. Substitute the definition of z_{ij} into the standardized model and collect terms to express each β_j (including β_0) in terms of γ, $\bar{x}_j$ and s_j. Then use your formulas to convert $\hat{\gamma}$ back, and check that you recover the $\hat{\beta}$ from Exercise 2, question 2 and the same predictions.

here your solution

```
# here your code
```

Exercise 4: How Noise Enters the Coefficients

[optional]

So far $\hat{\beta}$ has been a number we compute. Now treat it as an *estimate*: suppose the data really are generated by Equation (4), $\mathbf{Y} = \mathbf{X}\beta + \varepsilon$, for some fixed **true** β, where the entries of ε are independent with mean 0 and variance σ^2. Here ε is the true noise, not the residuals of a fit. A new draw of the noise gives a new $\mathbf{Y}$ and therefore a new $\hat{\beta}$.

Two facts you may use:

- $\mathbf{X}^\top\mathbf{X}$ is symmetric, so it has an eigendecomposition $\mathbf{X}^\top\mathbf{X} = \mathbf{V}\mathbf{D}\mathbf{V}^\top$ with orthonormal eigenvectors $\mathbf{v}_i$ (the columns of $\mathbf{V}$) and eigenvalues $d_i \geq 0$ on the diagonal of $\mathbf{D}$. When it is invertible, $(\mathbf{X}^\top\mathbf{X})^{-1} = \mathbf{V}\mathbf{D}^{-1}\mathbf{V}^\top$.
- For a fixed vector $\mathbf{a}$, $\mathrm{Var}(\mathbf{a}^\top\varepsilon) = \sigma^2\|\mathbf{a}\|^2$.

1. *[optional]* Substitute $\mathbf{Y} = \mathbf{X}\beta + \varepsilon$ into Equation (9) and show that

$$\hat{\beta} - \beta = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\varepsilon \qquad (10)$$

here your solution

2. *[optional]* Project the error onto an eigenvector $\mathbf{v}_i$ of $\mathbf{X}^\top\mathbf{X}$ and show that

$$\mathrm{Var}\left(\mathbf{v}_i^\top\hat{\beta}\right) = \frac{\sigma^2}{d_i} \qquad (11)$$

What does this say about the coefficients when two columns of $\mathbf{X}$ are nearly collinear?

> **Hint:** first show $\mathbf{v}_i^\top(\mathbf{X}^\top\mathbf{X})^{-1} = \frac{1}{d_i}\mathbf{v}_i^\top$, then use $\|\mathbf{X}\mathbf{v}_i\|^2 = \mathbf{v}_i^\top\mathbf{X}^\top\mathbf{X}\mathbf{v}_i$.

here your solution

3. *[optional]* Check Equation (11) by simulation on the Boston design matrix. Treat your $\hat{\beta}$ from Exercise 2, question 2 as the true β, and the residual standard deviation as σ. Draw new noise, build a new $\mathbf{Y}$, refit, and repeat 5000 times. Then compare the variance of the simulated coefficients along each eigenvector with σ^2/d_i.

```
# --- setup, written for you ---
sigma = np.std(Y - X @ beta, ddof=X.shape[1])
d, V = np.linalg.eigh(X.T @ X)   # eigenvalues in ascending order
rng = np.random.default_rng(0)

# one refit per row: Y_new = X @ beta + noise; rng.normal takes loc, scale
and size
```

```
# here your code

# --- comparison, written for you ---
var_sim = (beta_sims @ V).var(axis=0)
print(f"{'d_i':>12}  {'simulated':>12}  {'sigma^2/d_i':>12}")
for d_i, v_sim in zip(d, var_sim):
    print(f"{d_i:12.2f}  {v_sim:12.5f}  {sigma**2 / d_i:12.5f}")
```

4. *[optional]* Print the eigenvector with the smallest eigenvalue, `V[:, 0]`, next to the column names. Which columns of $\mathbf{X}$ does it combine, and why are they nearly collinear? Relate your answer to the size of the `nox` coefficient you saw in Exercise 3, question 2.

```
# here your code
```

here your solution